Ukrainian Catholic University

Bachelor Thesis

Autonomous Ground Navigation and Dynamic Obstacle Avoidance for Husky Robot in Pedestrian Areas

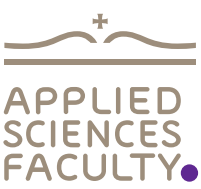
Author:
Radomyr Husiev

Supervisors:
Oleg Farenyuk
Oleksandr Kosovan

A thesis submitted in fulfillment of the requirements for the degree of Bachelor of Science

in the

Department of Computer Sciences and Information Technologies
Faculty of Applied Sciences



Lviv 2026

UKRAINIAN CATHOLIC UNIVERSITY

Faculty of Applied Sciences

Bachelor of Science

Autonomous Ground Navigation and Dynamic Obstacle Avoidance for Husky Robot in Pedestrian Areas

by Radomyr Husiev

Abstract

TODO

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Background and Motivation

Mobile robotic systems are an important field, especially for tasks that are monotonous, remote, or located in hazardous environments. Manual control of such robots is often infeasible due to the sheer number of systems and the volume of their tasks. Furthermore, physical barriers such as distance or signal interference can cause the remote control to fail. Therefore, autonomous systems are necessary to independently determine paths and navigate toward target locations.

Outdoor navigation is typically more complex than indoor navigation. Systems must handle less predictable conditions, such as moving obstacles, uneven terrain, and pedestrians who do not necessarily cooperate with the robot.

Today, ready-made solutions for autonomous navigation exist, such as the Nav2 stack in ROS [27]. However, modern outdoor systems often rely on resource-intensive sensors (like 3D LiDARs with data rates of millions of points per second, and RGB-D cameras) and complex algorithms (like Visual SLAM [36] and MPPI [40]). These approaches may provide superior accuracy in unstructured environments but demand substantial computational resources, large amounts of RAM (often 4-16 GB), and sometimes GPUs. This increases costs, drains battery life, and is not always possible on simple, low-cost platforms.

Therefore, there is a motivation to explore what can be accomplished using a minimal, lightweight sensor suite. By avoiding heavy computations, it is possible to build a more accessible and energy-efficient system.

Instead of generating a map from scratch using SLAM, existing semantic data such as OpenStreetMap (OSM) can be used for global routing [16]. Instead of processing 3D point clouds or images, a simple 2D LiDAR can handle immediate, dynamic obstacles like pedestrians or trees. Finally, instead of using visual odometry, the system can rely on standard GPS fused with an IMU and wheel odometry for localization [26].

Together, these lightweight components provide all the necessary data for a robot to navigate, albeit with lower spatial fidelity. This thesis aims to demonstrate that such a minimal setup allows for safe, real-time, collision-free point-to-point navigation in a semi-structured outdoor park environment, achieving high success rates while respecting strict CPU and memory bounds.

The setup includes a differential-drive ground robot in an outdoor park. Pedestrian paths provide a semi-structured, topologically known environment. Unlike road networks, pedestrian zones lack strict traffic rules or lane changes, significantly reducing the need for heavy semantic computer vision processing. However, unlike completely unstructured off-road environments [24], these paths have clear geometric boundaries that can be modelled and tracked. The system's objective is to navigate autonomously from a start point to a target point without human intervention.

1.2 Objectives

The primary objective of this thesis is to design, implement, and evaluate a resource-efficient autonomous navigation system for a UGV in semi-structured outdoor environments. Specifically, the research addresses the following goals:

- Develop global path planning using the A* search algorithm on an OpenStreetMap environment graph.
- Design obstacle detection by processing 2D LiDAR data to extract and classify geometric primitives (lines and circles).
- Implement local navigation by combining spatial localization (via an Extended Kalman Filter fusing GPS, IMU, and wheel odometry) with Timed Elastic Band trajectory optimization to follow the global path while avoiding obstacles.

The system was validated in a simulated environment, with performance evaluated based on success rate, safety margins, and travel time.

1.3 Structure Of The Thesis

The remainder of this thesis is structured as follows:

- **Chapter 2 (Related Work)** surveys existing approaches to path planning, localization, semantic mapping, and obstacle abstraction, highlighting the gap in lightweight solutions for outdoor navigation.
- **Chapter 3 (Theoretical Background)** details the foundational concepts, including sensor characteristics, robot kinematics, hierarchical navigation, trajectory optimization, sensor fusion, and geometric feature extraction.
- **Chapter 4 (Proposed Solution)** describes the hardware setup, system architecture, software implementation (including OSM processing, LiDAR abstraction, EKF localization, and custom TEB optimizer), safety mechanisms, and ethical considerations.
- **Chapter 5 (Experiments)**
- **Chapter 6 (Conclusion)**

Chapter 2

Related Work

2.1 Path and Trajectory Planning

The problem of autonomous navigation is classically divided into:

- Global path planning.
- Local trajectory planning and obstacle avoidance.

Research has explored both fields, developing various algorithms to handle different environmental constraints and vehicle kinematics [29].

For global planning, graph-based approaches like A* are commonly used because they are reliable and optimal on static maps [17].

However, navigating dynamic environments requires real-time local planning. Early methods, such as Artificial Potential Fields (APF) [18], modeled the environment as a physical field where the goal attracts and obstacles repel the robot. This approach often suffered from local minima and struggled with narrow gaps. Later, the Vector Field Histogram (VFH) [5] offered a simpler alternative with better performance, though it struggled with complex paths and kinematic constraints. Other approaches focused on dynamic environments by using collision cones to predict intersections between moving objects [10].

A widely used approach is the Dynamic Window Approach (DWA) [14]. It evaluates a set of feasible velocities for the next few seconds and chooses the best one based on goal proximity and obstacle avoidance. While DWA addresses vehicle kinematics and produces smooth motion, it struggles to generate paths in highly complex environments. As a result, it is included in the ROS Nav2 stack primarily as a simple baseline planner [27].

The Elastic Band approach [31] models the global path as a deformable band that stretches around newly detected obstacles. However, it only provides a geometric path rather than a complete trajectory, meaning it does not calculate the specific speeds required to drive the route.

The Timed Elastic Band (TEB) algorithm [34] extends this idea using a Model Predictive Control (MPC) approach. Unlike purely reactive methods, MPC-style algorithms predict and optimize the trajectory over a future time window while enforcing the vehicle's physical limits (kinodynamic constraints). TEB optimizes this trajectory to balance execution time and safe distance from obstacles. More recently, sampling-based MPC algorithms, such as Model Predictive Path Integral Control (MPPI), have been introduced for aggressive driving [40]. Due to its performance, Nav2 recently replaced TEB with MPPI as its advanced trajectory planner.

While MPC methods generate better trajectories than reactive methods like DWA, they require more computational power. MPPI, in particular, requires parallel sampling that often necessitates a GPU, making it unsuitable for constrained hardware.

Therefore, a CPU-optimized approach like TEB [33] provides a good balance between navigational performance and computational efficiency.

2.2 Localization

Accurate navigation requires reliable spatial localization. Currently, methods like Visual-Inertial SLAM (VI-SLAM) and 3D LiDAR-based SLAM are widely used. However, these methods are computationally heavy. For example, Schmidt et al. [36] showed that VI-SLAM algorithms can consume 4-6 GB of RAM, and their accuracy can degrade significantly when the frame rate is reduced to save processing power. Additionally, outdoor environments like parks often cause SLAM algorithms to drift or lose tracking entirely due to moving foliage and a lack of clear structural features.

Because of these limitations, using an Extended Kalman Filter (EKF) to fuse GPS, Inertial Measurement Units (IMU), and wheel odometry serves as a possible alternative. Moore and Stouch [26] showed that while wheel odometry drifts over time, fusing it with IMU data improves accuracy. Adding GPS, even if the signal is interfered by trees or other obstracles, further reduces long-term drift. This provides reliable localization without the heavy overhead of SLAM.

2.3 Semantic Mapping

SLAM is not only intended for localization but also for mapping. To further avoid the computational costs of SLAM, researchers have explored using existing semantic maps for global planning. Min et al. [24] note a growing trend of reducing SLAM dependency by navigating via pre-existing semantic data. Other studies highlight that explicit grid maps may not even be necessary, as internal visual odometry can sometimes provide high accuracy on its own [30].

OpenStreetMap (OSM) is a crowdsourced geographic database with tags describing road types, surface materials, and geometry. Hentschel and Wagner [16] showed that autonomous navigation can be successfully performed using OSM geodata. A distinct benefit of OSM over SLAM is its semantic context: it distinguishes footpaths from grass and provides surface details rather than just labeling space as free or occupied. Furthermore, Stahr et al. [38] use OSM attributes like path width to create boundaries for local planners, which can be used to limit the search space for trajectory optimization algorithms.

2.4 Obstacle Abstraction

Even with a lightweight map, processing raw sensor data for immediate obstacle avoidance remains resource-intensive. A 3D LiDAR produces a massive number of points and introduces the problem of filtering out the ground so the road itself is not treated as an obstacle [9]. This makes 3D LiDAR computationally expensive.

Using a 2D LiDAR solves the vertical complexity but still produces hundreds of points per scan. While simpler ultrasonic sensors exist, they struggle with angled surfaces and lack precise direction [4]. To optimize 2D LiDAR processing, point clouds are often abstracted into geometric primitives. Catapang and Ramos [8] and Nguyen et al. [28] grouped points within a specific distance into clusters. To extract shapes, Sezer and Gokasan [37] used circle approximations. For line extraction, Nguyen et al. [28] found that the "Split and Merge" algorithm is the most efficient method for converting raw LiDAR points into lines, while still being very accurate.

Dealing with dynamic agents like pedestrians is another challenge. While some approaches treat pedestrians differently from static obstacles, they often rely on heavy Reinforcement Learning and 3D tracking [12], which conflicts with computational efficiency. However, Arras et al. [2] demonstrated that detecting dynamic agents in 2D range data is possible using only static geometric features.

To further optimize feature detection, Xavier et al. [41] introduced an approach with an $O(n)$ linear complexity using Inscribed Angle Variance (IAV). This allows a system to quickly categorize features into straight lines, arcs, and legs within milliseconds.

2.5 Conclusion

The problem of autonomous navigation has been researched extensively, leading to a variety of path planning and obstacle avoidance algorithms. However, many modern advancements rely on resource-intensive methods, utilizing sensors like 3D LiDARs and requiring heavy algorithms like SLAM or MPPI. As a result, there is a limited understanding of the performance bounds of the minimal, low-cost sensor suites. Because advanced algorithms dominate recent studies, the capabilities of computationally lightweight architectures in unpredictable outdoor environments remain underexplored.

Based on these limitations, the goal of this research is to build and evaluate a resource-efficient navigation system. By replacing SLAM with semantic OpenStreetMap data, abstracting 2D LiDAR data into geometric obstacles, and using CPU-optimized TEB algorithms, this thesis aims to show that a minimal sensor configuration can demonstrate feasible autonomous navigation in complex outdoor environments.

Chapter 3

Theoretical Background

3.1 Sensor Characteristics

Autonomous navigation systems rely on multiple to perceive the environment and estimate the vehicle's state. Because a core objective of this research is resource efficiency, it is important to understand the capabilities and limitations of the minimal sensor suite:

- Wheel Encoders: Uses wheel encoders to estimate the robot's relative displacement by measuring the rotation of its wheels. While useful for short-term tracking, it accumulates drift over time due to mechanical slip, which is especially prominent in skid-steer Unmanned Ground Vehicle (UGV) configurations where wheel slip is significant [23].

- Inertial Measurement Unit (IMU): Contains accelerometers and gyroscopes to measure linear acceleration and angular velocity. It provides frequent motion updates but suffers from integration drift and bias, meaning small noise errors grow quadratically over time.

- Global Positioning System (GPS): Provides absolute geographic positioning (latitude, longitude) by trilaterating satellite signals. However, it is prone to high-frequency noise, low update rates, and signal degradation under trees or near tall structures [36].

- 2D LiDAR (Light Detection and Ranging): Uses laser pulses to measure distances in a single horizontal plane, outputting a point cloud in polar coordinates. While computationally much lighter than 3D LiDAR, it lacks vertical resolution and cannot directly capture vertical structure, such as distinguish between terrain features and vertical obstacles.

3.2 Differential Drive UGV

In this thesis, the robot uses a four-wheeled skid-steer differential drive configuration. While the machine has four points of contact with the ground, steering is achieved through a velocity differential between the left and right wheel pairs.

To generate realistic and executable trajectories, autonomous planners must respect the physical limitations of the robot, which are broadly categorized into kinematic and dynamic constraints. Together, these define the robot's kinodynamic constraints.

- Kinematic constraints. A differential drive system acts under non-holonomic kinematic constraints: it can rotate in place and move forward or backward, but

it cannot move laterally (it cannot slide sideways) without sliding. Therefore, the robot is controlled via its linear (v) and angular (ω) velocities [25]. If a planner generates a path that requires lateral movement, the hardware physically cannot execute it.

- Dynamic constraints. The UGV has physical mass and limited motor torque, so it cannot change velocities instantly. Therefore, dynamic constraints define the robot's maximum achievable velocities (v_{max}, ω_{max}) and maximum accelerations (a_{max}, ϵ_{max}).

Unlike two-wheeled systems, the four-wheeled configuration introduces lateral slip (skidding) during rotation, which increases drift in wheel-based odometry [23]. Thus, accurate sensor fusion becomes more important to correct for the drift.

3.3 Hierarchical Navigation Architecture

Classical autonomous navigation typically employs a hierarchical, two-level architecture. Planning a kinematically feasible, collision-free trajectory from a starting point to a distant goal in a single step is computationally too expensive. Therefore, the problem is split into global and local planning. The global planner operates on a static map to find a purely topological route to the destination, ignoring the vehicle's kinematics and dynamic obstacles. The local planner then acts upon a small sliding window of this global route, functioning in real-time and generating motion commands that avoid immediate, dynamic or previously unobserved obstacles.

3.4 Spatial Representation

3.4.1 Coordinate Systems

GPS sensors output data in geodetic coordinates (latitude, longitude, altitude). Trajectory planners operate in Cartesian coordinates. Geodetic coordinates must be projected onto a local tangent plane in order for the local planner to calculate distances in meters.

3.4.2 Graph-Based Map Representation

For a global planner to generate paths, it requires the environment to be abstracted into a directed semantic graph [16]. Unlike grid-based maps that treat the world as a matrix of occupied or free pixels, a semantic graph represents vertices as discrete spatial coordinates (such as path intersections), and edges as traversable paths (such as pedestrian roads). Complex paths made of multiple physical segments are simplified into single edges connecting these intersections. The weight of each edge is the sum of all its original segment lengths. This preserves distance accuracy while reducing the number of vertices the planning algorithm must evaluate.

The A* algorithm [15] navigates this graph. It calculates the shortest path by minimizing the function $f(n) = g(n) + h(n)$, where $g(n)$ is the exact cost from the start vertex to vertex n and $h(n)$ is the estimated heuristic cost. For A* to guarantee the mathematically shortest path, the heuristic must be admissible, meaning it must never overestimate the actual cost to the goal [32]. Because a straight line is always the shortest possible distance between two points, the Euclidean heuristic is admissible.

With the heuristic cost in place, the algorithm first processes paths closer to the goal, thus the frontier of the search stretches in an elliptical shape, pointing at the

target. This naturally reduces the number of vertices processed, while preserving accuracy.

3.5 Trajectory Optimization

The Timed Elastic Band (TEB) approach [34] formulates local navigation as a multi-objective optimization problem with a receding-horizon strategy. At each control step, the algorithm predicts a short-term trajectory over a finite time window, applies the first generated control command, and then continuously re-optimizes the trajectory as new sensor data arrives. The trajectory itself is defined as a sequence of intermediate poses between the start and goal. Each pose consists of x, y coordinates (in meters), θ (heading angle, in radians), and Δt (time interval, in seconds). The trajectory is initially generated as a sequence of points, with Δt and θ linearly interpolated.

The trajectory is then deformed based on constraints and obstacles. The problem is framed as a non-linear least squares problem to be optimized. The robot's poses act as the parameters. The physical and spatial constraints act as cost functions (also known as residuals). The optimization engine minimizes a weighted sum of these cost functions. In a least squares solver, the total cost is defined as $\frac{1}{2}\sum R_i^2$. Therefore, to apply a weight w to a cost, the raw error is multiplied by $\sqrt{w}$, ensuring the squared residual provides the strictly weighted cost.

The standard formulations of these residuals [34] ensure safety, kinematic feasibility, and efficiency:

3.5.1 Obstacle Cost

To prevent collisions, the solver penalizes poses that fall within the safety radius of geometric primitives. For a circular obstacle, the cost calculates the shortest distance from the obstacle center O to the robot line segment AB. P is the closest point on AB and is represented using $P = A + t(B - A)$. The projection scalar t is clamped to $[0, 1]$ to ensure the closest point lies strictly on the segment. Let $d = \|O - P\|$:

$$R_{obs} = \begin{cases} \sqrt{w} \cdot (r_{obs} + r_{safe} - d) & \text{if } (r_{obs} + r_{safe}) > d, \\ 0 & \text{otherwise.} \end{cases} \tag{3.1}$$

3.5.2 Kinematic Cost

To enforce the differential-drive constraint, the robot's motion between two configurations is modeled as an arc segment of constant curvature [34]. This requires the angle ϑ_i between the robot's heading and the displacement vector $\mathbf{d}_{i,i+1}$ at the start configuration to be equal to the corresponding angle ϑ_{i+1} at the final configuration. Rather than optimizing these angles directly, the constraint is formulated as a cross product:

$$\begin{pmatrix} \cos\theta_i \\ \sin\theta_i \\ 0 \end{pmatrix} \times \mathbf{d}_{i,i+1} = \mathbf{d}_{i,i+1} \times \begin{pmatrix} \cos\theta_{i+1} \\ \sin\theta_{i+1} \\ 0 \end{pmatrix}. \tag{3.2}$$

Simplified, the constraint reduces to a single scalar equation, yielding the resulting least-squares residual R_{kin}:

$$R_{kin} = \sqrt{w} \cdot \Big[(\cos\theta_i + \cos\theta_{i+1})\Delta y - (\sin\theta_i + \sin\theta_{i+1})\Delta x\Big]. \tag{3.3}$$

3.5.3 Dynamics Costs

This cost imposes penalties if the physical limits of a UGV are exceeded. Using finite differences, the linear velocity cost is formulated as:

$$R_v = \begin{cases} \sqrt{w} \cdot (|v| - v_{max}) & \text{if } |v| > v_{max}, \\ 0 & \text{otherwise.} \end{cases} \tag{3.4}$$

Analogous formulations are applied for linear acceleration, as well as angular acceleration and velocity, ensuring angle differences are normalized to $[-\pi, \pi]$. Two poses are required to calculate the finite differences for the velocity costs. Three poses are needed for acceleration costs, as they require two consecutive velocities to calculate the change in velocity.

3.5.4 Time Cost

To prevent the solver from minimizing other penalties by outputting infinite execution times, a penalty is applied directly to the Δt parameters. This cost forces the system to progress toward the goal:

$$R_t = \sqrt{w} \cdot \Delta t. \tag{3.5}$$

3.5.5 Solvers and Jacobians

Traditionally, the TEB algorithm models the trajectory optimization as a hyper-graph, where robot poses represent the vertices and the kinematic or spatial constraints represent the edges. However, this hyper-graph formulation is mathematically equivalent to a standard non-linear least-squares optimization problem. In a least-squares framework, the poses act as the parameter blocks, and the constraints act as the residual blocks.

To solve this objective function, optimization engines require Jacobians – matrices of partial derivatives indicating how changes in the trajectory parameters affect the total cost. These matrices are used to find the gradient of the cost functions and can be calculated analytically or numerically. Because trajectory constraints usually affect only adjacent poses, the resulting mathematical system is highly sparse. Solvers apply sparse factorization methods (such as Cholesky decomposition) to efficiently solve the equations, iteratively updating the parameters until they converge on a smooth, physically feasible trajectory.

3.5.6 Temporal Coherence

If an optimizer starts from a blank slate (a cold start) at every control step, it may fail to converge within strict time limits and may lead to inconsistent trajectories. MPC-style approaches, including TEB, solve this by utilizing a Warm Start [33]. Because the environment changes slightly between control frames, the optimized trajectory from the previous time step can be preserved and used as the initial guess for the next iteration. This temporal coherence reduces the number of iterations required for the solver to find a local minimum.

3.6 Sensor Fusion

Sensors contain noise and inherent inaccuracies. Wheel odometry accumulates drift from mechanical slippage, while IMU exhibits integration drift. Small offsets in IMU

sensor bias and noise result in quadratic position error growth and orientation errors caused by gravity leakage [26]. Global Positioning System (GPS) sensors provide absolute global coordinates but suffer from high-frequency noise and interference under tree cover or near structures [36].

The Kalman Filter fuses sensor streams into a single reliable state estimate [26]. It maintains a state vector and a covariance matrix, which represent the system's current uncertainty for each sensor. It uses IMU and wheel odometry to continuously predict the robot's next state, but as time passes, the uncertainty grows. GPS is used to correct the predicted state upon measurement and reduce uncertainty.

The standard Kalman Filter assumes linear system dynamics. However, mobile robots involve non-linear movements, such as turning and circular trajectories, which require trigonometric functions. The Extended Kalman Filter addresses this by linearizing motion and models at the current state estimate (achieved through Taylor series expansion) [26].

This fusion also ensures that even if a sensor like GPS becomes temporarily unavailable, the robot can continue to navigate using its internal motion model until the absolute signal is restored, though this dead reckoning fallback is inherently time-limited before accumulated drift exceeds safe operational thresholds [7].

3.7 Geometric Feature Extraction

2D LiDAR outputs data in polar coordinates (distance and angle). To be used by the optimization engine, this point cloud must be segmented and abstracted into geometric primitives (lines and circles) in Cartesian space.

3.7.1 Pre-processing and Clustering

Raw LiDAR scans frequently contain isolated noise spikes and artifacts. Therefore, the points are first passed through a median filter to reject extreme outliers. The filtered point clouds are then grouped into discrete clusters using distance-based segmentation [28]. Points are assigned to a single sequential object if the Euclidean distance between consecutive measurements P_i and P_{i+1} is below a defined break distance. The vehicle diameter, with some padding, can serve as an effective threshold, as narrower gaps would not be passable by the robot.

3.7.2 Shape Classification

Clusters must be categorized as linear or circular to apply the correct geometric extraction logic. For clusters with three or more points, a geometric pre-filter [41] is used to evaluate the curvature. A chord vector is drawn between the cluster's endpoints, P_1 and P_3, and the cross product is used to calculate the perpendicular arc depth to the median point, P_2. If this depth is bounded between 10% and 70% of the chord length, the cluster exhibits sufficient convexity to be classified as a circle. Clusters falling outside this range, or clusters exhibiting an excessive maximum internal point-to-point distance, are classified as lines.

3.7.3 Line Fitting

Line extraction uses the Split-and-Merge algorithm [28]. First, a chord is drawn between the cluster's endpoints. The algorithm calculates the perpendicular distance from every internal point to the chord. If the maximum distance exceeds a predefined

threshold, the cluster is split at that peak point into two sub-scans, and the process recurses. The result is a set of distinct linear segments that approximate complex shapes like walls or curbs.

3.7.4 Circle Fitting

For clusters identified as circular obstacles (such as trees or pedestrians), geometric parameters (center and radius) must be extracted. Algebraic methods, such as the circumcenter approach [41], attempt to find an analytical solution by selecting three points and computing the intersection of their perpendicular bisectors.

While mathematically exact for perfect curves, algebraic fitting frequently fails when applied to 2D LiDAR data. Because LiDAR only senses the front face of an obstacle, the returned points form a shallow arc. For distant or small obstacles, the limited resolution of the scanner causes these arcs to approach a straight line. Furthermore, because the laser beam has a physical width, reflections from the highly angled flanks of cylindrical objects register prematurely. This smearing effect flattens the perceived curve, causing traditional algebraic methods to systematically overestimate the cylinder's diameter by up to 19% [13].

Chapter 4

Proposed Solution

4.1 Problem Formulation

The objective of this research is to enable a UGV to reach a target position (defined via GPS coordinates or a relative Cartesian offset) from an initial starting point entirely without human intervention. The solution presents an OSM-based hierarchical autonomous navigation system operating in a semi-structured outdoor pedestrian environment. A semi-structured environment, such as a park, provides fixed paved paths but introduces dynamic, unpredictable obstacles such as pedestrians.

The core constraint of the problem is to design a resource-efficient setup using a minimal sensor suite (2D LiDAR, GPS, IMU, wheel odometry) without relying on computationally intensive and energy-demanding hardware like GPUs, 3D LiDARs, or RGB-D cameras. By combining semantic-graph-based global routing with receding-horizon local trajectory optimization, the system minimizes computational demands while maintaining reliable obstacle avoidance and spatial accuracy.

4.2 Hardware and Environment Setup

The experimental subject of this research is the Clearpath Husky A200 [11], a medium-sized UGV. The Husky is a rugged, four-wheeled skid-steer platform designed for research applications. It has a physical footprint of $99 \times 67 \times 39$ cm, weighing approximately 50 kg.

The UGV is equipped with a minimal sensor suite, which includes:

- Built-in Wheel Encoders, with the resolution of 78000 ticks/m.
- IMU:
- 2D LiDAR:
- GPS Receiver:
- Compute Unit: ROS2 Jazzy [22]

The proposed system was designed for Stryiskyi Park, Lviv, Ukraine. The environment is semi-structured, featuring mapped, paved pedestrian paths but lacking traffic rules or lane markings. While the park's topography includes both flat and hilly regions, this work is built for predominantly flat sections, as navigation on hilly terrains using only 2D sensors poses additional challenges in ground removal and obstacle filtering [19]. This environment poses several challenges:

- The park features dense tree coverage, which might degrade GPS signals, testing the resilience of the EKF sensor fusion.

- There are dynamic, non-cooperative entities – primarily pedestrians, bicycles, and pets – whose unpredictable trajectories test the responsiveness and real-time collision avoidance capabilities of the system.

4.3 System Architecture

The system consists of two ROS2 nodes:

- Controller Node: Executed directly on the UGV. It processes raw sensor data, extracts geometric primitives for obstacle abstraction, and computes both the global path and local trajectory. Running this node onboard minimizes network latency and ensures real-time responsiveness.
- Visualization Node: Operates off-board on a separate machine within the same local network. It subscribes to the Controller Node to render obstacles, paths, and trajectories, displaying them in a UI. It also allows entering the desired goal location or canceling navigation.

4.4 Software Implementation

4.4.1 Semantic Mapping and Global Routing

OSM is used for a semantic map. To process this data, the Python `osmnx` library [3] is utilized. It allows for the extraction of a multigraph for a specific OSM polygon relation (such as the park boundary), where traversable paths act as edges and intersections act as vertices. To filter out non-navigable terrain, the following edge tags are evaluated:

- `surface=`. Surfaces other than `paved, asphalt, concrete, paving_stones, sett, cobblestone` are excluded.
- `highway=`. For example, `motorway` and `cycleway` are excluded.
- `access=` and `foot=`. If any of these is `no` or `private`, the road is discarded.

After initial filtering, the global path can be generated. The A* algorithm is used to get a sequence of vertices that need to be followed in order.

The local planner does not operate on a graph; instead, it works in a Cartesian coordinate system. Therefore, the vertices sequence has to be converted into a sequence of 2D points connected by straight lines. Because physical footpaths are rarely perfectly straight, single edges in the graph are often represented in OSM as a sequence of smaller linear segments (stored within the `geometry` attribute). By splitting each edge into these underlying segments, the global planner outputs a sequence of 2D Cartesian waypoints suitable for the local planner.

4.4.2 Obstacle Abstraction

The local trajectory planner requires distinct geometric constraints rather than a dense point cloud. To achieve this, a custom perception pipeline is implemented to process 2D LiDAR scans into Cartesian geometric primitives based on the theoretical models described in Chapter 2.

Pre-processing and Clustering

Raw LiDAR scans are converted from polar to Cartesian coordinates. Points below a minimum range (rays reflected from the vehicle itself) are discarded. The scan is passed through a 1D median filter of size N to reject isolated noise. The points are then grouped sequentially, broken based on the break distance (the vehicle width plus a safety margin).

Shape Classification

Clusters with fewer than three points are automatically classified as circles. For larger clusters, if the maximum internal Euclidean distance exceeds the geometry split threshold, it is considered a line. For the remaining clusters, the Xavier heuristic [41] is applied. The arc depth is calculated, and if it falls between 10% and 70% of the chord length, the cluster is classified as a circle; otherwise, it is treated as a line.

Primitives Extraction

To ensure stability against sensor noise and bias introduced with algebraic curvature methods of circle extraction, a spatial heuristic is used. The maximum Euclidean distance between any two points in the cluster (the chord width) establishes an initial diameter estimate. To counteract sensor noise and the systematic cylinder overestimation identified by Forsman et al. [13], the radius (half diameter) is clamped between predefined physical minimum and maximum bounds. Instead of calculating a mathematical center from the arc, the visible middle point of the cluster is projected outward from the sensor origin by the estimated radius to approximate the true geometric centroid. Finally, the radius is expanded, so that all points in the cluster are enclosed. This method mitigates overestimations while implicitly providing a safety margin. Clusters classified as lines are processed using the recursive Split-and-Merge algorithm to yield distinct linear segments.

Virtual Boundaries

To restrict the UGV to the pedestrian path, virtual line obstacles are generated. While path width can be extracted from OSM tags, in practice, these tags are often unmapped. Instead, the implementation uses a default value of 4 meters. Parallel line barriers are placed on both sides of the target trajectory, creating a safe routing corridor for the local planner.

4.4.3 Localization and State Estimation

An Extended Kalman Filter (EKF) is implemented via the `robot_localization` ROS2 package. The EKF is configured to fuse data from the internal IMU and wheel encoders for continuous state estimation. It is used for the trajectory updates, so as not to introduce sudden shifts for the TEB local planner.

Another EKF is used to take into account low-frequency GPS data. It is projected onto a local Cartesian tangent plane and fused to reduce the drift. This second filter's output is used to update the direction toward the target, ensuring the vehicle reaches the exact destination.

In fallback scenarios where GPS is disabled or unavailable, the direction to the global goal is only updated using the first filter, which uses IMU and wheel odometry.

4.4.4 Local Trajectory Optimization

A custom Python implementation of the Timed Elastic Band (TEB) algorithm was developed, utilizing the `pyceres` library [35] for non-linear optimization. `pyceres` provides minimal Python bindings that expose the core functionality of the highly optimized C++ Ceres Solver [1], allowing the high-level logic to be orchestrated in Python while maintaining near-native execution speeds.

Local Goal Selection and State Representation

Because optimizing a trajectory all the way to the final destination is infeasible to compute in real time, the TEB planner operates on a sliding window. At each control loop, the UGV's current position is projected onto the global path (the closest waypoint to the current position is selected). The algorithm traverses the path forward until an accumulated length is equal to the predefined lookahead distance. This coordinate becomes the temporary local goal for the optimizer. If the local goal falls inside or too close to an obstacle, it is slid along the global path until a free space is found.

To ensure the solver remains stable, the trajectory resolution is dynamically resized between iterations. The distances between consecutive poses are calculated. If a distance is lower than a lower bound, the pose is pruned, and the Δt of the previous one is increased by the Δt of the current one. If the distance is larger than the upper bound, a new pose gets inserted in the middle of the other two, and the Δt is halved and spread across the inserted pose and the first of the two old ones.

Custom Formulations of TEB Residuals

While standard kinematic, dynamic, time, and circular obstacle costs (as defined in Section 2.5) were utilized in the system, specific custom formulations had to be derived for linear obstacles and start conditions to maintain solver stability in Pyceres.

Line Obstacle Cost Calculating the exact shortest distance between two line segments introduces non-differentiable edge cases. To solve this and maintain smooth Jacobians for the Ceres solver, a custom residual was derived. The minimum distance from the trajectory segment (endpoints A, B) to the obstacle segment (endpoints C, D) is approximated using a LogSumExp (Softmin) function [6] over the four endpoint-to-segment distances ($d_A \dots d_D$):

$$d_{min} = -\frac{1}{\alpha} \ln \left(\sum_{k=1}^{4} \exp(-\alpha \cdot d_k) \right), \tag{4.1}$$

$$R_{line} = \begin{cases} \sqrt{w} \cdot (r_{safe} - d_{min}) & \text{if } r_{safe} > d_{min}, \\ 0 & \text{otherwise.} \end{cases} \tag{4.2}$$

Starting Dynamics Cost For the first segment of the trajectory (from the current position to the first planned pose), there is no "previous" pose to reference – and as such, there cannot be three poses to calculate acceleration. If unconstrained, the solver might generate a trajectory that begins at a high velocity while the physical robot is stationary, implicitly demanding an infinite, impossible instantaneous acceleration.

To prevent this, a custom dynamic cost is implemented specifically for the starting position. It uses the robot's current velocity ($\vec{v}_{curr}, \omega_{curr}$), retrieved directly from the

state estimation sensors, to calculate the acceleration and angular acceleration costs for the first trajectory segment.

Jacobians and Ceres Problem Setup

The standard ROS implementation of TEB relies on C++ and the `g2o` library [21]. However, because the TEB objective function is fundamentally a sum of squared weighted residuals, it can be easily adapted into a general-purpose non-linear least-squares framework. For this custom Python architecture, Google's Ceres Solver [1] was chosen via the `pyceres` library.

While `g2o` uses predefined graph edges, Ceres allows for the manual definition of residual blocks. Because `pyceres` lacks the C++ compile-time automatic differentiation feature of the native Ceres library, the partial derivatives for all implemented residuals were calculated analytically and provided directly to the solver.

To ensure the optimization can run in real-time, an obstacle filter is applied before constructing the Ceres problem. Rather than evaluating every obstacle against every pose, the system pre-calculates distances and only adds residual blocks to the solver for obstacles posing an immediate collision threat to specific segments. All obstacle parameters are pre-extracted into arrays upon initialization, allowing for vectorized operations instead of slow iterative loops.

The problem is solved using the Sparse Normal Cholesky linear solver via `pyceres`. Due to Python limitations, specifically the Global Interpreter Lock (GIL) [39], performance drops when increasing the number of threads, so the optimization is strictly restricted to a single thread. To maintain a predictable control rate, the solver is capped at a strict maximum number of iterations. The Sparse Normal Cholesky solver was selected because the TEB optimization problem produces a highly sparse Jacobian matrix, which this decomposition solves efficiently, as also demonstrated in the original TEB formulation [33].

Latency Compensation

The implementation uses a warm start approach to maintain temporal coherence. However, because non-linear optimization introduces computational latency, the UGV physically moves while the solver runs. If the trajectory state was preserved blindly, the initial poses would be outdated by the time the next frame begins.

To resolve this, a latency compensation mechanism is implemented. Before the next optimization step begins, the system queries the most recent odometry estimate (filtered through EKF). The displacement $(\Delta x, \Delta y, \Delta\theta)$ that accumulated during the optimization is used to transform the trajectory matrix. This ensures the warm-started trajectory matches the robot's current pose before being passed to the solver. After these adjustments, the first pose in the trajectory is translated into linear (v) and angular (ω) velocities and published to the UGV's motor controllers.

4.4.5 Parameter Selection

To ensure the autonomous system operates safely and reliably, the algorithmic parameters were tuned to match both the physical properties of the UGV and the computational constraints of the onboard hardware.

Spatial and Obstacle Constraints

- *Safety and Critical Radii.* The system uses a dual-boundary approach. The "safety radius" is set to 1.3 m, establishing a soft buffer slightly larger than the UGV's length. The optimizer penalizes trajectories entering this zone. Additionally, a "critical stop radius" is defined at 0.5 m. If any obstacle breaches this distance, an immediate emergency stop is triggered.

- *Cluster Break Distance.* Set to 1.4 m, which is roughly 1.5 times the physical width of the robot. If the gap between two sequential LiDAR points is smaller than this threshold, the UGV would physically struggle to navigate through it. Therefore, the clustering algorithm treats the gap as unpassable and merges the points into a single solid obstacle.

- *LiDAR Filtering.* A "min scan range" of 0.33 m is enforced to filter out laser rays reflecting off the Husky's own chassis. Additionally, a 1D median filter of size 3 is applied to the raw scans. This eliminates isolated single-ray noise (e.g., sensor artifacts or flying debris) without overly smoothing out genuine narrow obstacles.

- *Virtual Barriers.* To prevent the local planner from navigating off the pavement into unstructured terrain, a "barrier offset" of 2.0 m is applied. Empirically, pedestrian paths in the targeted park environment have a minimum width of roughly 4 meters. Applying a 2-meter virtual line obstacle to both the left and right of the global path's center confines the trajectory to the pavement.

Kinodynamic Limits

- *Velocity and Acceleration.* The maximum linear velocity (v_{max}) is restricted to 0.5 m/s, and angular velocity (ω_{max}) to 0.75 rad/s. While the Husky is capable of higher speeds, limiting velocity prevents severe skid-steering drift and increases safety in a crowded environment. Linear acceleration (a_{max}) is limited to 0.1 m/s^2. This low acceleration ensures smooth motion and provides a more predictable behavior for pedestrians.

Computational Constraints

- *Trajectory Lookahead.* The "max trajectory distance" is set to 7 m. This provides enough foresight for the TEB algorithm to navigate around large obstacles without overloading the solver with distant, irrelevant waypoints.

- *Trajectory Resolution.* The trajectory resolution limits are bounded between 0.5 m and 2.0 m. This means that each segment of the trajectory is at least 0.5 m and at most 2.0 m long. This ensures the matrix size remains small enough for real-time calculation while maintaining enough pose density to accurately describe curves.

- *Maximum Solver Iterations.* The Ceres optimizer is capped at 5 iterations per control frame. While the original TEB implementation [33] utilized four outer loops of five iterations each, computing all 20 iterations within a single cycle introduces significant latency, especially when using Python. Delaying the control response to calculate a mathematically perfect trajectory poses a safety risk in dynamic environments. Instead, the system executes only a single loop of 5 iterations and outputs this intermediate solution immediately to maintain

a high control rate. Because the algorithm employs a warm start strategy, the subsequent control frame continues refining the trajectory exactly where the previous one left off. This approach effectively distributes the optimization workload across control frames, ensuring real-time responsiveness without sacrificing long-term path convergence.

4.4.6 Safety Mechanisms

Because the system relies on a minimal sensor suite, it inherently possesses perceptual blind spots – most notably the lack of vertical vision due to the 2D LiDAR. To ensure the safe operation of a heavy UGV in a public environment, a hierarchy of safety nets was implemented. If any of the following conditions are met, the system overrides the local planner and triggers an immediate software-level emergency stop (E-Stop), halting all movement.

Terrain Anomalies

A 2D LiDAR scans a single horizontal plane and cannot detect low obstacles (like rocks), drop-offs, or curbs. To compensate, the internal IMU monitors the vehicle's orientation. If the pitch or roll exceeds a threshold, it indicates the UGV is driving over an unseen obstacle or falling into a ditch, triggering an immediate stop.

Proximity Breach

If the geometric abstraction pipeline detects any obstacle within a critical stop distance, the system assumes an imminent or ongoing collision and stops.

Stale Sensors

The controller monitors the timestamps of incoming LiDAR and odometry streams. If the sensors lag or timeout, the system stops to prevent the robot from executing outdated trajectories while driving blind.

Trajectory Collision Prediction

The TEB optimizer may occasionally fail to converge on a safe path within the time limit. Before sending commands to the motors, the controller evaluates the first K segments ("trajectory collision lookahead") of the generated trajectory. If any of these segments intersect with an obstacle, the trajectory is deemed unsafe.

Path Deviation

If the UGV is forced to dodge multiple obstacles and its position deviates more than X m (virtual corridor + padding, meaning that other safety features such as curb detection did not activate) from the global A* route, it is considered lost.

Localization Jumps

While GPS may produce high-frequency errors, IMU and wheel odometry do not. If a jump in the latter is noticed (significantly larger than the vehicle's maximum velocity), the system halts to wait for the localization to stabilize.

Motion Stall

If the controller is actively publishing velocity commands, but the sensors report little to no actual speed for a continuous period of time, the system deduces that the UGV is mechanically stuck and stops attempting to drive forward.

Network and Operator Override

The UGV operates autonomously but must remain under the ultimate supervision of a human operator using the off-board Visualization node. The UI publishes a continuous heartbeat signal over the local network. If the controller does not receive this signal for 5 seconds, it assumes the network connection has dropped. The robot automatically halts to ensure the operator never loses the ability to trigger a remote software cancellation.

4.5 Ethical and Safety Considerations

Deploying a 50 kg autonomous vehicle in a public space involves inherent safety risks. The primary ethical consideration is to ensure that the UGV does not pose a physical threat to pedestrians, pets, or property. As discussed, the system is limited in its perception due to its minimal sensor suite. To mitigate the risk, the system design prioritizes safety over navigational efficiency. This is reflected in the strict velocity caps, the critical stop radius, and the extensive software-level safety mechanisms described in 4.4.6. Quantitative benchmarking involving human-like dynamic agents is only conducted in simulation to eliminate physical risk.

4.6 Experimental Methodology

To evaluate the performance and resource efficiency of the proposed lightweight navigation architecture, a series of controlled quantitative experiments was conducted. While the system is designed for and qualitatively tested on physical hardware, the primary comparative benchmarking was executed within a simulation environment. Simulation allows for exact reproducibility, precise dynamic obstacle scripting, and isolated performance profiling without the physical constraints of battery degradation or unmeasured environmental hazards.

4.6.1 Simulation Environment Setup

The experiments were conducted using Gazebo Sim (version 8.9.0) [20]. To accurately reflect the semantic and spatial challenges of the target environment, a simulated representation of Stryiskyi Park was utilized. The environment's global topology was driven by OSM data, while the physical terrain was modeled using a 2D satellite projection overlaid on a topographic heightmap.

Because the core objective of the system is local obstacle avoidance, artificial geometric obstacles were injected into the simulated park environment along the known OSM paths. These obstacles were sized to approximate real-world hazards:

- Static Obstacles: Represented by 0.5 m-diameter stationary cylinders, simulating obstacles such as standing pedestrians or unmapped static objects.
- Dynamic Obstacles: Represented by cylinders moving via predefined kinematic trajectories at speeds between 0.5 m/s and 1.2 m/s, simulating walking pedestrians.

To ensure the simulation accurately reflected the physical constraints of the Clearpath Husky A200, realistic sensor noise was modeled. The simulated 2D LiDAR was constrained to the hardware's actual maximum range, and the simulated wheel odometry and IMU data were processed through the same EKF node as the physical robot.

4.6.2 Test Scenarios

The system was evaluated across several navigational scenarios, each designed to stress different aspects of the planner:

- Static Obstacle Course: The UGV navigated a 50-meter path populated with stationary obstacles arranged in offset patterns, requiring the TEB planner to execute smooth kinodynamic avoidance maneuvers.
- Dynamic Intersection: The UGV traversed a crossing where simulated dynamic agents (pedestrians) intersect the robot's planned global path perpendicularly, testing the planner's reaction time and emergency stop safety nets.
- Narrow Corridor: The UGV navigated a tightly bounded virtual OSM corridor (4 m wide) containing static obstacles, forcing the optimizer to be precise.

4.6.3 Baseline Comparison

To assess the performance of the proposed architecture, it was benchmarked against the industry-standard ROS2 Nav2 stack [27]. Nav2 provides two primary local planner options: DWA [14] and MPPI [40]. Because MPPI is highly resource-intensive and often requires GPU acceleration [24], DWA was used as the baseline.

To ensure a fair comparison, Nav2 was configured to operate under the identical lightweight sensor constraints. Rather than utilizing SLAM or heavy visual odometry, Nav2 was provided with a static occupancy grid corresponding to the OSM path limits and relied on its local costmap (populated by the 2D LiDAR) for obstacle detection. Both systems were given the same sequence of global waypoints generated by the A* graph search, isolating the comparison strictly to the efficiency and safety of the local trajectory execution.

4.6.4 Evaluation Metrics

The experimental evaluation focused exclusively on the performance of the local trajectory optimization and obstacle avoidance systems. The global routing component (the A* graph search over the OSM network) was excluded from benchmarking. A* is a deterministic, mathematically complete, and optimal algorithm [15] – given an admissible heuristic, it is proven to find the optimal path if one exists. Therefore, evaluating its success provides no insight. Instead, the metrics were designed to evaluate how efficiently and safely the system executed that predefined global path in the presence of dynamic, unmapped constraints.

Quantitative data was collected over $N = 30$ independent simulated runs for each test scenario to evaluate the performance of the proposed system against the Nav2 baseline. The data was analyzed using descriptive statistics.

Primary Navigation Metrics

- Success Rate (%): The percentage of experimental runs where the UGV successfully navigated from the start point to within 1 meter of the final target

destination. A run was classified as a failure if the UGV collided with an obstacle, triggered an unrecoverable safety emergency stop, or exceeded a timeout limit.

- Path Efficiency (Ratio): Measured as the shortest-path distance generated by the global A* planner divided by the actual distance traveled by the UGV. A score of 1.0 implies the UGV followed the center of the path perfectly.
- Mean Travel Time (s): The average time required to complete successful runs. The standard deviation was also calculated to evaluate the consistency of the planner's behavior.

Resource Efficiency Metrics

Because the work prioritizes computational accessibility, resource consumption is an important benchmark. Both the average values (mean) and the consistency (standard deviation, 5th percentile lows) were evaluated. During the simulation runs, system telemetry was logged at 1 Hz intervals to measure:

- CPU Utilization (%)
- Memory Footprint (MB)
- Control frame frequency (Hz): The number of control commands generated and sent by the algorithm per second. This one was measured after every control loop.

Chapter 5

Experiments and Results

Chapter 6

Conclusions

Bibliography

[1] Sameer Agarwal, Keir Mierle, and The Ceres Solver Contributors. *Ceres Solver*. 2012. URL: http://ceres-solver.org.

[2] Kai O. Arras, Oscar Martinez Mozos, and Wolfram Burgard. "Using Boosted Features for the Detection of People in 2D Range Data". In: *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*. 2007, pp. 3402–3407. DOI: 10.1109/ROBOT.2007.363998.

[3] Geoff Boeing. "OSMnx: New Methods for Acquiring, Constructing, Analyzing, and Visualizing Complex Street Networks". In: *Computers, Environment and Urban Systems* 65 (2017), pp. 126–139. DOI: 10.1016/j.compenvurbsys.2017.05.004.

[4] Johann Borenstein and Yoram Koren. "Obstacle avoidance with ultrasonic sensors". In: *IEEE Journal on Robotics and Automation* 4.2 (1988), pp. 213–218. DOI: 10.1109/56.2085.

[5] Johann Borenstein and Yoram Koren. "Real-time obstacle avoidance for fast mobile robots". In: *IEEE Transactions on Systems, Man, and Cybernetics* 19.5 (1989), pp. 1179–1187. DOI: 10.1109/21.44033.

[6] Stephen Boyd and Lieven Vandenberghe. *Convex Optimization*. Cambridge: Cambridge University Press, 2004. DOI: 10.1017/CBO9780470011515.

[7] Martin Brossard, Axel Barrau, and Silvère Bonnabel. "AI-IMU Dead-Reckoning". In: *IEEE Transactions on Intelligent Vehicles* 5.4 (2020), pp. 585–595. DOI: 10.1109/TIV.2020.2980758.

[8] Angelo Nikko Catapang and Manuel Ramos. "Obstacle Detection using a 2D LIDAR System for an Autonomous Vehicle". In: *Proceedings of the IEEE 6th International Conference on Control System, Computing and Engineering (ICCSCE)*. 2016, pp. 441–445. DOI: 10.1109/ICCSCE.2016.7893614.

[9] Marcello Cellina, Matteo Corno, and Sergio Matteo Savaresi. "LiDAR-Based Vehicle Detection and Tracking for Autonomous Racing". In: *arXiv preprint arXiv:2501.14502* (2025).

[10] Animesh Chakravarthy and Debasish Ghose. "Obstacle avoidance in a dynamic environment: a collision cone approach". In: *IEEE Transactions on Systems, Man, and Cybernetics, Part A: Systems and Humans* 28.5 (1998), pp. 562–574. DOI: 10.1109/3468.709600.

[11] Clearpath Robotics. *Husky A200 User Manual*. Accessed: 2026-04-23. URL: https://clearpathrobotics.com/husky-unmanned-ground-vehicle-robot.

[12] Yan Du et al. "Group Surfing: A Pedestrian-Based Approach to Sidewalk Robot Navigation". In: *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*. 2019, pp. 4418–4424. DOI: 10.1109/ICRA.2019.8793608.

[13] Mona Forsman et al. "Bias of cylinder diameter estimation from ground-based laser scanners with different beam widths: A simulation study". In: *ISPRS Journal of Photogrammetry and Remote Sensing* 135 (2018), pp. 84–92. DOI: 10.1016/j.isprsjprs.2017.11.013.

[14] Dieter Fox, Wolfram Burgard, and Sebastian Thrun. "The dynamic window approach to collision avoidance". In: *IEEE Robotics & Automation Magazine* 4.1 (1997), pp. 23–33. DOI: 10.1109/100.580977.

[15] Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. "A Formal Basis for the Heuristic Determination of Minimum Cost Paths". In: *IEEE Transactions on Systems Science and Cybernetics* 4.2 (1968), pp. 100–107. DOI: 10.1109/TSSC.1968.300136.

[16] Michael Hentschel and Bernardo Wagner. "Autonomous Robot Navigation Based on OpenStreetMap Geodata". In: *Proceedings of the IEEE Conference on Emerging Technologies and Factory Automation (ETFA)*. 2010, pp. 1–4. DOI: 10.1109/ETFA.2010.5625092.

[17] Karthik Karur et al. "A survey of path planning algorithms for mobile robots". In: *Vehicles* 3.3 (2021), pp. 448–468. DOI: 10.3390/vehicles3030027.

[18] Oussama Khatib. "Real-Time Obstacle Avoidance for Manipulators and Mobile Robots". In: *The International Journal of Robotics Research* 5.1 (1986), pp. 90–98. DOI: 10.1177/027836498600500106.

[19] Jihoon Kim, Hyungjin Jeong, and Donghun Lee. "Single 2D lidar based follow-me of mobile robot on hilly terrains". In: *Journal of Mechanical Science and Technology* 34 (2020), pp. 3025–3034. DOI: 10.1007/s12206-020-0835-7.

[20] Nathan Koenig and Andrew Howard. "Design and use paradigms for Gazebo, an open-source multi-robot simulator". In: *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. Vol. 3. IEEE, 2004, pp. 2149–2154. DOI: 10.1109/IROS.2004.1389727.

[21] Rainer Kümmerle et al. "g2o: A General Framework for Graph Optimization". In: *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*. 2011, pp. 3607–3613. DOI: 10.1109/ICRA.2011.5979949.

[22] Steven Macenski et al. "Robot Operating System 2: Design, architecture, and uses in the wild". In: *Science Robotics* 7.66 (2022), eabm6074. DOI: 10.1126/scirobotics.abm6074. URL: https://www.science.org/doi/abs/10.1126/scirobotics.abm6074.

[23] Anthony Mandow et al. "Experimental Kinematics for Wheeled Skid-Steer Mobile Robots". In: *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2007, pp. 1222–1227. DOI: 10.1109/IROS.2007.4399139.

[24] Chen Min et al. "Autonomous Driving in Unstructured Off-Road Environments: How Far Have We Come?" In: *Journal of Field Robotics* (2024). DOI: 10.1002/rob.70154.

[25] Javier Minguez, Florent Lamiraux, and Jean-Paul Laumond. "Motion Planning and Obstacle Avoidance". In: *Springer Handbook of Robotics*. Springer, 2008, pp. 827–852. DOI: 10.1007/978-3-540-30301-5_36.

[26] Thomas Moore and Daniel Stouch. "A Generalized Extended Kalman Filter Implementation for the Robot Operating System". In: *Proceedings of the 13th International Conference on Intelligent Autonomous Systems (IAS)*. Springer, 2015, pp. 335–348. DOI: 10.1007/978-3-319-08338-4_25.

[27] Nav2 Project. *Nav2 Controller Plugins*. 2024. URL: https://docs.nav2.org/configuration/index.html#controller-plugins (visited on 05/15/2024).

[28] Viet Nguyen et al. "A Comparison of Line Extraction Algorithms Using 2D Laser Rangefinder for Indoor Mobile Robotics". In: *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2005, pp. 1929–1934. DOI: 10.1109/IROS.2005.1545234.

[29] Anish Pandey, Shweta Pandey, and Dayal R. Parhi. "Mobile robot navigation and obstacle avoidance techniques: A review". In: *International Journal of Control, Automation and Systems* 15.2 (2017), pp. 968–982. DOI: 10.1007/s12555-014-0456-z.

[30] Ruslan Partsey et al. "Is Mapping Necessary for Realistic PointGoal Navigation?" In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2022, pp. 17232–17241.

[31] Sean Quinlan and Oussama Khatib. "Elastic bands: Connecting path planning and control". In: *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*. 1993, pp. 802–807. DOI: 10.1109/ROBOT.1993.291936.

[32] Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. 4th ed. Pearson, 2020.

[33] Christoph Rösmann et al. "Efficient Trajectory Optimization Using a Sparse Model". In: *Proceedings of the European Conference on Mobile Robots (ECMR)*. 2013, pp. 138–143.

[34] Christoph Rösmann et al. "Trajectory Modification Considering Dynamic Constraints of Autonomous Robots". In: *Proceedings of the 7th German Conference on Robotics (ROBOTIK)*. 2012, pp. 1–6.

[35] Paul-Edouard Sarlin, Philipp Lindenberger, and Shaohui Liu. *pyceres: Minimal Python bindings for the Ceres Solver*. https://github.com/cvg/pyceres. 2025.

[36] Fabian Schmidt et al. "Visual-Inertial SLAM for Unstructured Outdoor Environments: Benchmarking the Benefits and Computational Costs of Loop Closing". In: *Journal of Field Robotics* (2025). DOI: 10.1002/rob.22581.

[37] Volkan Sezer and Metin Gokasan. "A novel obstacle avoidance algorithm: "Follow the Gap Method "". In: *Robotics and Autonomous Systems* 60.9 (2012), pp. 1123–1134. DOI: 10.1016/j.robot.2012.05.021.

[38] Pascal Stahr, Jochen Maaß, and Henner Gärtner. "Application of Path Planning for a Mobile Robot Assistance System Based on OpenStreetMap Data". In: *Robotics* 12.4 (2023), p. 113. DOI: 10.3390/robotics12040113.

[39] Guido Van Rossum and Fred L. Drake. *Python 3 Reference Manual*. Scotts Valley, CA: CreateSpace, 2009. ISBN: 1441412697.

[40] Grady Williams et al. "Aggressive Driving with Model Predictive Path Integral Control". In: *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*. 2016, pp. 1433–1440. DOI: 10.1109/ICRA.2016.7487277.

[41] João Xavier et al. “Fast Line, Arc/Circle and Leg Detection from Laser Scan Data in a Player Driver”. In: *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*. 2005, pp. 3930–3935. DOI: 10.1109/ROBOT.2005.1570721.